

Zadanie: Sieci Neuronowe i Propagacja Wsteczna

Rozwiązanie Zadania

18 grudnia 2025

Architektura Sieci

- **Wejście:** $x = [x_1, x_2]$
 - **Warstwa ukryta:** 2 neurony, funkcja aktywacji ReLU
 - **Wyjście:** 1 neuron, funkcja liniowa (brak aktywacji)
 - **Funkcja straty:** Mean Squared Error (MSE)
 - **Dane:** $x = [2, 3]$, $y_{target} = 5$
-

1 Część 1: Obliczenie pochodnych (Inicjalizacja 1.0)

Założenia początkowe: Wszystkie wagi (w) oraz biasy (b) w sieci są zainicjalizowane wartością 1.0.

1.1 Propagacja w przód (Forward Pass)

Warstwa ukryta:

Obliczenie sumy ważonej (z) oraz aktywacji (h) dla obu neuronów (ze względu na identyczne wagi, obliczenia są takie same dla neuronu 1 i 2):

$$z = (x_1 \cdot w_1) + (x_2 \cdot w_2) + b = (2 \cdot 1) + (3 \cdot 1) + 1 = 6$$

$$h = \text{ReLU}(6) = 6$$

Warstwa wyjściowa:

$$y_{pred} = (h_1 \cdot w_{out1}) + (h_2 \cdot w_{out2}) + b_{out}$$

$$y_{pred} = (6 \cdot 1) + (6 \cdot 1) + 1 = 13$$

Błąd (MSE):

$$L = (y_{pred} - y_{target})^2 = (13 - 5)^2 = 8^2 = 64$$

1.2 Propagacja wsteczna (Backward Pass)

Gradient błędu względem predykcji:

$$\frac{\partial L}{\partial y_{pred}} = 2(y_{pred} - y_{target}) = 2(13 - 5) = 16$$

Oznaczmy $\delta_{out} = 16$.

A. Gradienty dla warstwy wyjściowej

- Dla wag wyjściowych (w_{out}):

$$\frac{\partial L}{\partial w_{out}} = \delta_{out} \cdot h = 16 \cdot 6 = \mathbf{96}$$

- Dla biasu wyjściowego (b_{out}):

$$\frac{\partial L}{\partial b_{out}} = \delta_{out} \cdot 1 = \mathbf{16}$$

B. Gradienty dla warstwy ukrytej

Pochodna funkcji ReLU dla $z = 6$ wynosi 1. Błąd propagowany do warstwy ukrytej:

$$\delta_h = \delta_{out} \cdot w_{out} \cdot f'(z) = 16 \cdot 1 \cdot 1 = 16$$

- Dla wag połączonych z x_1 (w_{11}, w_{21}):

$$\frac{\partial L}{\partial w_{x1}} = \delta_h \cdot x_1 = 16 \cdot 2 = \mathbf{32}$$

- Dla wag połączonych z x_2 (w_{12}, w_{22}):

$$\frac{\partial L}{\partial w_{x2}} = \delta_h \cdot x_2 = 16 \cdot 3 = \mathbf{48}$$

- Dla biasów ukrytych (b_1, b_2):

$$\frac{\partial L}{\partial b} = \delta_h \cdot 1 = \mathbf{16}$$

2 Część 2: Analiza inicjalizacji zerami

Pytanie: Czy sieć jest w stanie się uczyć, gdy wszystkie parametry zostaną zainicjalizowane z wartością 0.0?

Odpowiedź: Nie, sieć nie będzie w stanie uczyć się poprawnie.

Uzasadnienie obliczeniowe:

1. **Forward Pass:** Wszystkie sumy ważone wyniosą 0. Wyjście sieci $y_{pred} = 0$.
2. **Backward Pass:**
 - Gradient na wyjściu: $\delta_{out} = 2(0 - 5) = -10$.
 - Pochodna dla wag wyjściowych: $\delta_{out} \cdot h = -10 \cdot 0 = 0$. (Wagi wyjściowe nie są aktualizowane).
 - Pochodna dla warstw ukrytych: Zależy od iloczynu z wagami wyjściowymi ($\dots w_{out} \cdot \dots$). Skoro $w_{out} = 0$, gradient zanika i nie dociera do warstw niższych.

Nawet jeśli bias wyjściowy ulegnie zmianie, wagi pozostaną zerowe lub (w przypadku symetrycznej inicjalizacji inną stałą) neurony w warstwie ukrytej będą uczyć się identycznych funkcji, co prowadzi do problemu symetrii.

3 Część 3: Pytania teoretyczne

3.1 1. Zastosowanie sieci neuronowych vs. If/Else

Zastosowanie sieci: Sieci neuronowe najlepiej sprawdzają się w zadaniach, w których dane są nieustrukturyzowane (obrazy, dźwięk), a reguły są zbyt złożone dla jawnych wzorów (np. Computer Vision, NLP).

Dlaczego nie “If-Else”? Ręczne pisanie reguł jest niemożliwe w złożonych problemach z dwóch powodów:

1. **Eksplzja kombinatoryczna:** Liczba warunków potrzebna do opisanie np. wyglądu kota na zdjęciu dąży do nieskończoności.
2. **Brak generalizacji:** Sztywne warunki (np. `if x1=5 then y=5.6`) nie potrafią interpolować wyników dla nowych danych (np. `x1=5.1`).

3.2 2. Rola funkcji aktywacji

Funkcje aktywacji (np. ReLU, Sigmoid) wprowadzają do sieci **nieliniowość**.

Brak funkcji aktywacji: Jeśli usuniemy funkcje aktywacji, sieć staje się matematycznie równoważna **pojedynczej warstwie liniowej** (regresji liniowej). Złożenie funkcji liniowych jest nadal funkcją liniową, więc sieć traci zdolność rozwiązywania złożonych problemów.

3.3 3. Rola Dropout’u

Dropout to technika regularyzacji zapobiegająca przeuczeniu (overfitting). Polega na losowym “wyłączaniu” neuronów podczas treningu. Wymusza to na sieci tworzenie nadmiarowych reprezentacji wiedzy i zapobiega poleganiu neuronów na pojedynczych ścieżkach sygnału.